Q, K, V Projection with cuBLAS - Part 5 Video Notes

Video: Computing Query, Key, Value vectors using cuBLAS. Keep `projectBf16RowMajor` and `prefill` in `src/main.cpp` open while reading. RMSNorm explains where the input comes from; RoPE explains what happens to Q and K afterward.

The host snippets describe the Part 5 milestone (`42532af`), before full layers, generation, and batching. Use `git show 42532af:src/main.cpp` for that snapshot. The projection helper's matrix-layout reasoning still applies to the current engine.

Where we are in the engine

Embedding gather gave every token a 2,048-value starting vector. Layer 0's input RMSNorm produced `normalized_gpu`, also `[T, 2048]`, where `T` is the number of prompt tokens. Now we turn each normalized token vector into three **different views of the same current hidden state**:

```
normalized hidden states X [T, 2048]
  ├─ learned Wq -> queries Q [T, 2048] -> 32 heads × 64
  ├─ learned Wk -> keys    K [T,  512] ->  8 heads × 64
  └─ learned Wv -> values  V [T,  512] ->  8 heads × 64
```

The words “query,” “key,” and “value” are names for the jobs these vectors will have in attention. They are **activations calculated for this prompt**. `weights.w_q[0]`, `weights.w_k[0]`, and `weights.w_v[0]` are the fixed learned matrices used to make them.

Why not compare the embedding vectors directly?

My rough notes use “Amazing food” as an intuition. For the token representing “food,” “Amazing” might be useful context, even if the two starting embedding vectors are not especially similar. A plain dot product of raw embeddings only measures one kind of similarity. Attention lets the model learn separate ways to ask a question and to describe what each earlier token offers.

- A **query** is an analogy for what a token is looking for.
- A **key** is an analogy for what a token can be matched on.
- A **value** is the information mixed into the output after the matches are scored.

I say “analogy” because a Q coordinate is not an English-language question, and a K coordinate is not a literal label. The projections are learned numerical transformations. Q and K make attention **directional**: the score for query token `i` looking at key token `j` need not equal the score for query `j` looking at key `i`. Later, the causal mask makes that direction especially important.

The “clean windows” versus “downloaded Windows” example is another way to feel the motivation. Surrounding words can change the useful context for a token. Attention is the mechanism that lets a token combine information from earlier tokens; the Q/K/V projections set up that mechanism. These examples do not imply that any particular head is guaranteed to recognize a specific English relationship.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

What does “projection” mean here?

For one normalized hidden vector `x` with 2,048 values, each output coordinate is a dot product with **one row of a learned weight matrix**:

```
q[o] = sum over h=0..2047 of x[h] * Wq[o, h]
k[o] = sum over h=0..2047 of x[h] * Wk[o, h]
v[o] = sum over h=0..2047 of x[h] * Wv[o, h]
```

For all `T` tokens together, with row-major tensors:

```
Q = X × Wqᵀ       X [T, 2048], Wq [2048, 2048], Q [T, 2048]
K = X × Wkᵀ       X [T, 2048], Wk [ 512, 2048], K [T,  512]
V = X × Wvᵀ       X [T, 2048], Wv [ 512, 2048], V [T,  512]
```

This is a matrix multiplication, not a vocabulary lookup and not yet the dot product between **different tokens**. Each token's row can be projected independently at this stage. The model's printed architecture marks these linear layers `bias=False`, and this helper does not add a bias.

Tiny fake example, just to separate projection from attention: let `x = [2, 1]` and a toy query matrix have rows `[1, 0]` and `[0, 2]`. Then `q = [2, 2]`. A different toy key matrix with rows `[1, 1]` and `[1, -1]` gives `k = [3, 1]` **for the same token**. The matrices choose different learned combinations of the input features. The real model uses 2,048 input features and the trained weight values from SafeTensors.

Why 32 Q heads but only 8 K and V heads?

The Q output width is `2048 = 32 × 64`. K and V each have width `512 = 8 × 64`. Splitting a projected row into consecutive 64-value chunks gives the heads:

```
Q token row: [Q head 0 | Q head 1 | ... | Q head 31]
K token row: [K head 0 | K head 1 | ... | K head 7]
V token row: [V head 0 | V head 1 | ... | V head 7]
```

This is **grouped-query attention** (GQA). Four neighboring query heads share one K head and one V head:

```
Q heads 0-3   -> K/V head 0
Q heads 4-7   -> K/V head 1
...
Q heads 28-31 -> K/V head 7

kv_head = query_head / 4
```

The code sets `QUERY_HEADS_PER_KV_HEAD` from the constants and uses that division in the next attention step. The projection step only creates the differently sized Q, K, and V buffers; it does not copy a K or V head four times. Fewer K/V heads also mean fewer K/V values to cache when decode is implemented.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The host-side buffers and three calls

`prefill` allocates one GPU buffer for Q and two smaller GPU buffers for K and V:

```
q_bytes  = token_count * HIDDEN_SIZE * sizeof(__nv_bfloat16);
kv_bytes = token_count * KV_DIM      * sizeof(__nv_bfloat16);

cudaMalloc(&q_gpu, q_bytes);
cudaMalloc(&k_gpu, kv_bytes);
cudaMalloc(&v_gpu, kv_bytes);
```

`HIDDEN_SIZE` is `2048`; `KV_DIM` is `512`. For example, with `T = 3`, Q has `3 × 2048` BF16 values, while K and V each have `3 × 512`. The input `normalized_gpu` is already on the GPU from RMSNorm, and the model loader already made direct pointers to the three weight matrices. There is no CPU round trip between the operations.

The host creates a cuBLAS handle and calls the same helper three times:

```
projectBf16RowMajor(handle, normalized_gpu, weights.w_q[0], q_gpu, T, 2048);
projectBf16RowMajor(handle, normalized_gpu, weights.w_k[0], k_gpu, T,  512);
projectBf16RowMajor(handle, normalized_gpu, weights.w_v[0], v_gpu, T,  512);
```

Those lines show the idea; the source uses `cublas_handle`, `token_count`, `HIDDEN_SIZE`, and `KV_DIM` and checks all three returned statuses. The current milestone runs **layer 0 only**. The weight struct has arrays for all 16 layers, but the forward pass has not yet looped over them.

Why use cuBLAS instead of one thread per output?

Unlike embedding gather, every projected output needs a sum over 2,048 input values. With `T` tokens and hundreds or thousands of output columns, that is a large matrix multiplication. cuBLAS supplies tuned GPU GEMM routines for this job. `GEMM` means general matrix multiplication, and `cublasGemmEx` computes:

```
C = alpha * op(A) * op(B) + beta * C
```

Here `alpha = 1`, `beta = 0`, so we want only the new multiplication result. The weight and input are BF16, `CUBLAS_COMPUTE_32F` requests FP32 computation, and the output buffer is BF16. The result is rounded when stored; the whole pipeline is therefore not magically FP32 end to end. NVIDIA's cuBLAS documentation defines the operation, types, and leading dimensions.

The row-major versus column-major puzzle

This is the part that made me stop and draw the matrices. Our buffers are physically **row-major**: a token's features sit next to each other. The classic cuBLAS GEMM interface interprets matrix storage as **column-major**. We do not physically transpose giant GPU buffers just to call it. Instead, we use the fact that the same flat bytes read in column-major look like a transposed matrix.

We want $Y = X \times W^T$, with:

```
X: [T, H]      W: [O, H]      Y: [T, O]
H = 2048       O = 2048 for Q, 512 for K/V
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The row-major result `Y` has the same flat memory layout as a column-major Y^T of shape `[O, T]`. Transposing the equation gives $Y^T = W \times X^T$. In `projectBf16RowMajor`, cuBLAS receives the **weight as A** and the **input as B**:

```
flat W, read column-major, looks like W^T; CUBLAS_OP_T turns it back into W
flat X, read column-major, looks like X^T; CUBLAS_OP_N leaves it as X^T
cuBLAS result: W [O,H] × X^T [H,T] = Y^T [O,T]
flat output bytes: exactly our wanted row-major Y [T,O]
```

So the actual call uses `CUBLAS_OP_T`, `CUBLAS_OP_N`, and `(m, n, k) = (O, T, 2048)`. Its leading dimensions are `lda = 2048` for weights, `ldb = 2048` for input, and `ldc = O` for output. The leading dimension is the distance, in **elements**, between adjacent columns from cuBLAS's point of view; it is not a byte count. The transpose flags describe cuBLAS's view of the buffers, which is why reading just the flags without the storage layout feels backward.

Then RoPE, but only for Q and K

After the three projections, `prefill` calls `launchRope` on `q_gpu` with projection width `2048`, and on `k_gpu` with width `512`. Both use head width `64`, the position IDs, and the precomputed sine/cosine tables. `v_gpu` is left alone. RoPE changes Q/K **activations in place**, so the later Q·K comparisons contain relative-position information.

The host checks cuBLAS return statuses and CUDA launch errors, then synchronizes to catch GPU execution errors. In the Part 5 snapshot, `q_gpu`, `k_gpu`, and `v_gpu` are temporary prefill buffers freed before the function returns. The current engine keeps projection scratch in persistent `InferenceBuffers` for reuse across calls. In this part of the series, the projections' next consumer is the layer-0 self-attention code described in the Part 6 self-attention notes.

What to keep straight while watching

```
Wq/Wk/Wv = fixed learned weight matrices loaded from the model file
Q/K/V    = new prompt-dependent activations produced by matrix multiplication
Q/K      = rotated by RoPE before token-to-token scores
V        = later mixed using attention probabilities, never rotated here
```

The output of this video is a set of three projected buffers. It has not yet decided which earlier token matters most or mixed any values. That is what the attention calculation does next.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.